

HOMEWORK 1

Arthur Augusto Vieira Pinho Gimenez (579859)

Caio Felipe dos Santos Oliveira (516882)

Roberto Walace Santos da Silva (581562)

30 de setembro de 2026

Sumário

Questão 1	2
Implementação em R	3
Questão 2	4
Item 1	4
Item 2	5
Definições	5
Cálculo de 10 observações	6
Cálculo de 300 observações	7
Item 3	8
Definições	8
Cálculo de 10 observações	9
Cálculo de 300 observações	11
Item 4	12
Item 5	14
Questão 3	15
Definições	15
Item 1	15
Item 2	17
Item 3	19
Item 4	21
Questão 4	21
Item 1	22
Item 2	24
Item 3	25
Item 4	26

Questão 1

A primeira questão solicita que uma amostra de 300 observações do conjunto de dados em anexo, armazenado em formato .csv, seja carregada pelo R e armazenada no objeto `data_group`. No entanto, é notável que o arquivo *HW1_bike_sharing* apresenta 731 observações, portanto precisamos extrair apenas 300 do total, seguindo o critério requisitado pela questão.

Considere M como o maior número de matrícula entre os integrantes do grupo, ou seja, $M = \max\{579859, 516882, 581562\} = 581562$. Defina r como a posição inicial das 300 observações, cuja fórmula é:

$$r = 1 + (M \bmod 100)$$

Substituindo o valor de M na fórmula acima,

$$r = 1 + (581562 \bmod 100)$$

$$r = 63$$

Defina f como a posição final dos dados extraídos, segue, por aritmética simples:

$$300 = f - 63 + 1$$

$$f = 362$$

Com base nessas duas posições, é possível determinar a data da primeira e última observação no arquivo .csv, conforme solicitado na questão.

Tabela 1: Linhas da posição 63 e 362

	instant	dteday	season	weathersit	temp	casual	registered
63	63	2011-03-04	1	2	10.7	214	1730
362	362	2011-12-28	1	1	12.3	255	2047

Portanto, ao analisar as linhas acima, é conclusivo que as datas referentes à observação 63 e à 362 são respectivamente *2011-03-04* e *2011-12-28*.

Implementação em R

```
1      # Instala e carrega o pacote para leitura de arquivos .csv
2      install.packages("readr")
3      library(readr)
4
5      # Registra os nomes das colunas
6      nomes_colunas <- c("", "instant", "dteday", "season",
7                          "weathersit", "temp", "casual", "registered")
8
9      # Extrai a amostra de 300 observações
10     data_group <- read_csv("HW1_bike_sharing.csv", skip = 62,
11                           n_max = 300)
12     colnames(data_group) <- nomes_colunas
```

Código 1: Extração de amostra

O código acima é responsável por extrair as 300 observações da fonte de dados. As funções *install.packages()* e *library()* presentes nas linhas 2-3 são respectivamente responsáveis por instalar uma biblioteca e carregá-la no interpretador do R.

- **readr**: É uma biblioteca que fornece funções rápidas para ler dados retangulares, como CSV ou TSV.

Para extrair a parcela dos dados é preciso pular a linha referente aos nomes das colunas, portanto, nas linhas 6-7 esses nomes são gravados antecipadamente em formato de vetor no objeto *nomes_colunas*.

- $c(p_1, p_2, \dots, p_n)$: É uma função que retorna um vetor ou uma lista a partir dos parâmetros passados.

Na linha 10, a função *read_csv(file, skip, n_max)* é responsável pela leitura do arquivo *HW1_bike_sharing.csv* e extração dos dados, armazenando-os no objeto *data_group*. Na linha seguinte, a função *colnames()* recebe *data_group* como parâmetro e armazena os nomes das colunas. Segue definição dos argumentos de *read_csv*:

- **file**: O nome do arquivo.
- **skip**: Número de linhas puladas antes da leitura dos dados.
- **n_max**: Número máximo de linhas a serem lidas.

Questão 2

A segunda questão requisita o tratamento inicial dos dados, como identificação dos tipos, cálculo das medidas de tendência central e a criação de histograma e box-plot. Antes disso, no entanto, solicitou a definição de uma variável `total_user` formada pela soma dos usuários casuais e registrados. Para isso, somamos os dois vetores que armazenam cada variável, como demonstrado no código a seguir.

```
13 # Questão 2
14 total_user = data_group[, "casual", drop = TRUE] +
    data_group[, "registered", drop = TRUE]
```

Código 2: Definição de `total_user`

Nós acessamos toda a coluna do `data_group` por `[,nome]`, enquanto o parâmetro `drop = "TRUE"` força a conversão de coluna de `data.frame` para vetor.

Item 1

Inicialmente, o item pede a classificação das variáveis quanto a serem numéricas ou categóricas, informando a unidade de medida caso numérica e as categorias caso categóricas.

- **instant** (numérica): Quantifica a posição da observação na lista, portanto é uma variável numérica.
- **dteday** (numérica): A data, como representada na tabela, indica um número que quantifica um instante do tempo, logo é uma variável numérica.
 - A data (YYYY-MM-DD) é composta por três unidades de medida: Ano, mês e dia.
- **season** (categórica): Associa a cada número (1-4) uma classe de *"estação do ano"*, portanto é categórica.
 - As categorias são: "Primavera", "verão", "outono" e "inverno".
- **weathersit** (categórica): As condições meteorológicas também são associadas a números, mas cada um representa uma classe, portanto é uma variável categórica.
 - As categorias são: "Céu limpo", "Nublado", "Chuva fraca" e "Chuva forte"
- **temp** (numérica): A temperatura é uma variável que quantifica diretamente a quantidade de energia térmica associada a algo, portanto é uma variável numérica.
 - A unidade de medida é em graus Celsius normalizada por 41, conforme descrito na questão.

- **casual** (numérica): Quantifica o número de usuários casuais, portanto é uma variável numérica.
- **registered** (numérica): Quantifica o número de usuários registrados, portanto é uma variável numérica.

O item também solicita a verificação da existência de valores ausentes e sua quantificação, caso necessário. Para isso, foi criado o seguinte trecho de código em R:

```
16 # Item 1.
17 anyNA((data_group)) # Retorno no terminal (FALSE)
```

Código 3: Verificação de valores ausentes

A linha 15 do código apresenta a função `anyNA()`, que verifica se há pelo menos um valor ausente presente no argumento passado (vetor, list, data.frame...). O retorno no terminal foi *FALSE*, logo não há valores ausentes.

Item 2

O item solicita o cálculo das medidas de tendência central (média, mediana e moda) para as variáveis numéricas relevantes, que são `temp`, `casual` e `registered`.

Definições

def - Média: Um valor numérico que quando multiplicado pela quantidade de dados resulta na soma dos valores do conjunto.

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (1)$$

def - Mediana: O valor central de um conjunto qualquer de dados **ordenados**.

$$Med = X_{(N+1)/2} \quad (2)$$

Caso existam dois valores centrais, isto é, o resultado foi um número decimal, a mediana é dada por

$$Med = \frac{X_{\lfloor (N+1)/2 \rfloor} + X_{\lceil (N+1)/2 \rceil}}{2} \quad (3)$$

def - Moda: O valor que mais se repete em um conjunto de dados qualquer.¹

¹Caso não se repita valores, a distribuição é amodal. Duas modas é bimodal, enquanto três ou mais é multimodal.

Cálculo de 10 observações

Conforme requisitado na tarefa, segue o cálculo manual das 10 primeiras observações do conjunto de dados e os resultados do código em R.

Tabela 2: 10 primeiras observações

temp	10.7	15.8	15.4	10.7	12.0	12.1	16.0	13.0	13.5	15.8
casual	214	640	114	244	316	191	46	247	724	982
registered	1730	1437	491	1628	1817	1700	577	1730	1408	1435

- Médias:

$$\bar{x}_{\text{temp}} = \frac{1}{10} \cdot (10.7 + 15.8 + 15.4 + 10.7 + 12.0 + 12.1 + 16.0 + 13.0 + 13.5 + 15.8)$$
$$\bar{x}_{\text{temp}} = 13.5$$

$$\bar{x}_{\text{casual}} = \frac{1}{10} \cdot (214 + 640 + 114 + 244 + 316 + 191 + 46 + 247 + 724 + 982)$$
$$\bar{x}_{\text{casual}} = 371.8$$

$$\bar{x}_{\text{registered}} = \frac{1}{10} \cdot (1730 + 1437 + 491 + 1628 + 1817 + 1700 + 577 + 1730 + 1408 + 1435)$$
$$\bar{x}_{\text{registered}} = 1395.3$$

- Medianas:

Para calcular as medianas é necessário ordenar os dados da segunda tabela, ao se ordenar, é obtido:

Tabela 3: Dados ordenados

temp	10.7	10.7	12.0	12.1	13.0	13.5	15.4	15.8	15.8	16.0
casual	46	114	191	214	244	247	316	640	724	982
registered	491	577	1408	1435	1437	1628	1700	1730	1730	1817

Pela equação (3) e dados da tabela acima,

$$Med_{\text{temp}} = \frac{13.0 + 13.5}{2} = 13.25$$
$$Med_{\text{casual}} = \frac{244 + 247}{2} = 245.5$$
$$Med_{\text{registered}} = \frac{1437 + 1628}{2} = 1532.5$$

- Modas:

$$Moda_{temp} = \text{Bimodal}$$

$$Moda_{casual} = \text{Amodal}$$

$$Moda_{registered} = 1730$$

Para verificar o resultado no R, o seguinte trecho de código foi escrito:

```

19   # Item 2.
20   mean(data_group[1:10, "temp", drop = TRUE])
21   mean(data_group[1:10, "casual", drop = TRUE])
22   mean(data_group[1:10, "registered", drop = TRUE])
23
24   median(data_group[1:10, "temp", drop = TRUE])
25   median(data_group[1:10, "casual", drop = TRUE])
26   median(data_group[1:10, "registered", drop = TRUE])
27
28   install.packages("DescTools") # Possui uma função para a moda
29   library(DescTools)
30
31   Mode(data_group[1:10, "temp", drop = TRUE])
32   Mode(data_group[1:10, "casual", drop = TRUE])
33   Mode(data_group[1:10, "registered", drop = TRUE])

```

Código 4: Cálculo das 10 observações (1)

Foram utilizadas três funções para calcular cada tipo de medida de tendência central para cada variável. Nas linhas 26-27 é instalado e carregado o pacote `DescTools`, que possui funções relacionadas à estatística descritiva, como para o cálculo da moda.

- **mean**: Calcula a média aritmética de um conjunto de dados (linhas 18-20).
- **median**: Calcula a mediana de um conjunto de dados (linhas 22-24).
- **Mode**: Calcula a moda de um conjunto de dados, retornando NA caso a distribuição seja amodal, o valor atômico caso haja uma única moda e um vetor de modas para 2 ou mais (linhas 29-31).

Os resultados exibidos no console foram iguais aos obtidos calculando manualmente, portanto as definições matemáticas utilizadas pelo R são a mesma.

Cálculo de 300 observações

```

35     # 300 observações
36     mean(data_group[, "temp", drop = TRUE])
37     mean(data_group[, "casual", drop = TRUE])
38     mean(data_group[, "registered", drop = TRUE])
39
40     median(data_group[, "temp", drop = TRUE])
41     median(data_group[, "casual", drop = TRUE])
42     median(data_group[, "registered", drop = TRUE])
43
44     Mode(data_group[, "temp", drop = TRUE])
45     Mode(data_group[, "casual", drop = TRUE])
46     Mode(data_group[, "registered", drop = TRUE])

```

Código 5: Cálculo das 300 observações (1)

Os 9 resultados obtidos no console podem ser visualizados na tabela a seguir:

Tabela 4: Medidas de tendência central - 300 observações

	temp	casual	registered
Média	22.116	786.7867	3023.62
Mediana	22.2	691.5	3238.5
Moda	26.0	Polimodal	Polimodal

Ao interpretar os dados da tabela para a temperatura, como a média e mediana estão próximas, certamente há uma maior frequência de temperaturas próxima a 22°C. Para os usuários casuais e registrados, é incerto dizer se a média ou mediana é mais adequada para interpretação do valor central sem o conhecimento de outliers, medidas de dispersão ou formato da distribuição.

Item 3

O item solicita o cálculo dos quartis e do intervalo interquartil para uma das variáveis relevantes. Além disso, pede para determinar o limite inferior e superior, quantificar os outliers e identificar suas respectivas datas.

Definições

def - Primeiro quartil (Q1): É definido como o número na posição equivalente à 25%.

$$Q1 = X_{(N+1)/4}$$

def - Segundo quartil (Q2): Tem a definição equivalente à da mediana (2).

def - Terceiro quartil (Q3): É definido como o número na posição equivalente à 75%.

$$Q1 = X_{3(N+1)/4}$$

Em caso do cálculo dos quartis resultar em número racional, o quartil será dado pela média aritmética dos elementos na posição piso e teto, análogo ao especificado na equação (2).

def - Intervalo interquartil (*IQR*): Intervalo de dados que estão limitados pelos quartis.

$$IQR = Q3 - Q1$$

def - Limite superior (L_s): Limite superior para dados típicos.

$$L_s = Q3 + 1.5 \cdot IQR$$

def - Limite inferior (L_i): Limite inferior para dados típicos.

$$L_i = Q1 - 1.5 \cdot IQR$$

Cálculo de 10 observações

A variável escolhida para o cálculo foi `temp`, analisando os valores da tabela (3), decorre:

$$\begin{aligned} Q1 &= \frac{10.7 + 12.0}{2} = 11.35 \\ Q2 &= \frac{13.0 + 13.5}{2} = 13.25 \\ Q3 &= \frac{15.8 + 15.8}{2} = 15.8 \end{aligned}$$

Calculando o intervalo interquartil e os limites

$$\begin{aligned} IQR &= 15.8 - 11.35 = 4.45 \\ L_s &= 15.8 + 1.5 \cdot 4.45 = 22.475 \\ L_i &= 11.35 - 1.5 \cdot 4.45 = 4.675 \end{aligned}$$

Segue código para verificação dos valores no R:

```
48 #Item 3.
49 quartis10 <- quantile(data_group[1:10, "temp", drop = TRUE],
50                        probs=c(0.25,0.50,0.75), type = 6)
51 IQR10 <- unname(quartis10[3]-quartis10[1])
52 Ls10 <- unname(quartis10[3]) +1.5*IQR10
53 Li10 <- unname(quartis10[1]) - 1.5*IQR10
```

```
quartis10; IQR10; Ls10; Li10 # Exibe os valores no console
```

Código 6: Cálculo das 10 observações (2)

O código acima calcula os quartis utilizando a função `quantile(x, probs, type...)`, retornando um vetor nomeado com os quartis em ordem. Os parâmetros principais são:

- `x`: Um vetor que contém os dados para o cálculo dos quartis.
- `probs`: Um vetor que contém as posições dos quartis, proporcionalmente de 0 a 1.
- `type`: Indica o tipo de cálculo de quartis que será utilizado, o tipo 6 é o que mais se aproxima das definições dadas nessa seção

Depois o intervalo interquartil e os limites foram calculados com a expressão respectiva em R (linhas 48-50), utilizando a função `unname()` para retornar um valor simples (sem nome).

O resultado do console e comparação com o cálculo manual é exibido na tabela a seguir:

Tabela 5: Comparação dos resultados

10 observações	Q_1	Q_2	Q_3	IQR	L_s	L_i
Cálculo manual	11.350	13.250	15.800	4.45	22.475	4.675
Código R	11.675	13.250	15.800	4.125	21.9875	5.4875

Ao analisar a tabela, é verificável que houve uma discrepância entre os valor do primeiro quartil, o que ocasionou discrepância nos valores que são dependentes. Isso ocorreu por que o R, quando calcula o quartil, ao invés de escolher o elemento do piso e do teto e calcular a média aritmética, ele faz uma interpolação linear com base no valor decimal de $p_1 = (N + 1)/4$ e $p_3 = 3(N + 1)/4$. Defina p como uma posição genérica do quartil e d sua parte decimal, podemos calcular um quartil genérico como:

$$Q_n = x_{\lfloor p \rfloor} + d \cdot (x_{\lceil p \rceil} - x_{\lfloor p \rfloor})$$

Alternativamente, para dados positivos

$$Q_n = x_{\lfloor p \rfloor} + (p - \lfloor p \rfloor) \cdot (x_{\lceil p \rceil} - x_{\lfloor p \rfloor})$$

Podemos fazer o cálculo manual utilizando a equação acima para verificar se o resultado é o mesmo:

$$Q_1 = 10.7 + (2.75 - 2) \cdot (12 - 10.7)$$

$$Q_1 = 11.675$$

De fato, o resultado é igual ao do R, como exibido na tabela (5). Além disso, ao se verificar manualmente, observa-se que não houve outliers. Também cabe informar que variável *temp* foi escolhida por que é uma variável física que geralmente recebe tratamento estatístico.

Cálculo de 300 observações

Como o método utilizado para cálculo de quartis do R geralmente é considerado mais preciso estatisticamente, não há problema utilizá-lo para a amostra inteira. O código utilizado foi:

```
56   #Amostra inteira
57   quartis <- quantile(data_group[, "temp", drop = TRUE],
58                       probs=c(0.25,0.50,0.75), type = 6)
59   IQR <- unname(quartis[3]-quartis[1])
60   Ls <- unname(quartis[3]) +1.5*IQR
61   Li <- unname(quartis[1]) - 1.5*IQR
62   quartis; IQR; Ls; Li # Exibe os valores no console
```

Código 7: Cálculo das 300 observações (2)

Os valores obtidos estão exibidos na tabela a seguir:

Tabela 6: Comparação dos resultados

300 observações	Q1	Q2	Q3	IQR	L_s	L_i
Código R	16.425	22.200	27.900	11.475	45.1125	-0.7875

A questão também solicitou a quantificação das observações atípicas e identificação de suas datas. Para isso, foi criada uma função personalizada em R chamada `encontrar_outliers`:

```
64   encontrar_outliers <- function(data_group, ls, li){
65     datas <- data_group[, "dteday", drop = TRUE]
66     temperatura <- data_group[, "temp", drop = TRUE]
67
68     data_temp <- data.frame(NA, NA)
69     k <- 1; i <- 1
70     while(i<length(temperatura)){
71       if(temperatura[i] > ls || temperatura[i] < li){
72         data_temp[k, "dteday"] <- datas[i]
73         data_temp[k, "temp"] <- temperatura[i]
74         k <- k + 1
75       }
76       i <- i+1
77     }
78     return(data_temp)
79   }
80   data_temp <- encontrar_outliers(data_group, Ls, Li)
```

Código 8: Determinação de outliers

A função é definida na linha 65, recebendo o `data_group`, o limite superior e o limite inferior como parâmetros. Nas linhas 66-67 são extraídos os vetores de temperatura e as datas do `data.frame`. Depois é inicializado um `data.frame` `data_temp` com valor em duas colunas de NA. O `while`, definido na linha 7, verifica para cada elemento se a temperatura é maior que o limite superior ou menor que o limite inferior, caso verdade, armazena a temperatura e a data no `data_temp`. Por fim, a função retorna o `data_temp` com as informações da data e temperatura (linha 76).

Para determinar os outliers relativos ao `data_group`, a função é chamada (linha 82), armazenando o valor no objeto `data_temp`. Ao ver os valores armazenados no objetos, é verificável que apenas possuía NAs, ou seja, não há outliers.

Quanto à relevância dos dados da tabela (6), o intervalo interquartil, o limite superior e o limite inferior conferem uma análise mais resistente à valores discrepantes, evitando, por exemplo, resultados imprecisos obtidos apenas pela aquisição da média e desvio-padrão. Além disso, os quartis permitem determinar o formato da distribuição, se é simétrica ou assimétrica ou se há uma densidade maior em algum quartil.

Item 4

Esse item solicita a construção de um boxplot e histograma e a análise deles quanto à forma, dispersão e outras características. Segue o código do boxplot em R e sua explicação.

```
83   # Box-Plot
84   boxplot(data_group[, "temp", drop = TRUE],
85           main = "Distribuição de Temperaturas", # Título
86           ylab = "Temperatura (C)", # Legenda no eixo y
87           col = "#81C784",           # Cor da caixa
88           border = "#2E7D32",        # Cor da borda
89           medcol = "#D32F2F",         # Cor da mediana
90           medlwd = 3,                 # Espessura da mediana
91           boxwex = 0.35,              # Largura da caixa
92           frame.plot = FALSE,         # Remove a moldura externa
93           outline = TRUE)             # Desenha outliers (se houver)
```

Código 9: Criação de boxplot

O primeiro parâmetro que a função `boxplot()` recebe é o vetor com os dados (observações), que nesse caso são de temperatura. Os parâmetros seguintes estão descritos de forma concisa com um comentário ao lado do código. Ao se exportar, a seguinte imagem foi obtida.

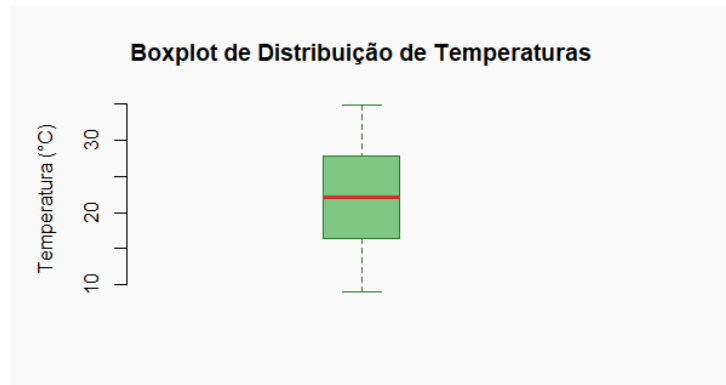


Figura 1

Ao analisar a figura (1), é conclusivo que o boxplot é simétrico, pois apresenta mediana central e hastes com aproximadamente o mesmo tamanho. Quanto à forma, o box plot parece implicar uma distribuição próxima à normal. Quanto à dispersão, o gráfico aparentemente apresenta uma dispersão padrão, nem muito concentrado nem muito disperso em relação aos valores centrais. Por fim, como afirmado anteriormente, os dados não apresentam outliers.

Segue o código do histograma em R.

```

95 # Histograma
96 hist(data_group[, "temp", drop = TRUE],
97      main = "Histograma de Distribuição de Temperaturas",
98      xlab = "Temperatura (C)", # Legenda no eixo x
99      ylab = "Frequência absoluta",
100     col = "#81C784",
101     border = "#2E7D32")

```

Código 10: Criação de histograma

O histograma apresenta os mesmos parâmetros do boxplot. Ao se exportar, a seguinte imagem foi obtida.

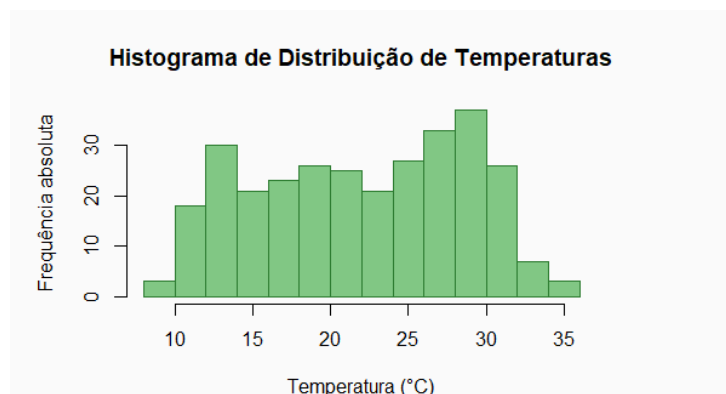


Figura 2

O histograma apresenta uma concentração ligeiramente maior de frequências em temperaturas superiores, mas também identifica um pico de frequências em torno dos 13 graus,

uma vantagem em relação ao box-plot, que não permite a identificação de distribuições com múltiplos picos. O boxplot apresentaria vantagem da demarcação de outliers se o conjunto de dados os apresentassem.

Quanto as medidas estatísticas calculadas nos itens 2 e 3, é verificável que a mediana e os quartis correspondem aos valores representados no boxplot, enquanto a média é melhor verificável pelo histograma pela altura média das barras, que parecem indicar que o resultado de 22.116 graus é razoável. A moda de 26 graus também está próxima da região de maior pico do histograma, corroborando sua credibilidade.

Item 5

Esse item solicita a criação de uma variável binária `low_usage` para classificar cada dia como baixa utilização caso o `total_user` respectivo seja menor que o primeiro quartil. Além disso, requisitou o valor de $Q1$, o número e a proporção de dias classificados como baixa utilização. A resolução está no código a seguir.

```
103   #Item 5.
104   Q1_total_user <- unname(quantile(total_user,
105                                   probs=c(0.25,0.50,0.75), type = 6))[1]
106   low_usage <- as.integer(total_user < Q1_total_user)
107   quantidade <- as.numeric(sum(low_usage == 1))
108   proporcao <- quantidade/300
```

O primeiro quartil de `total_user` é calculado na linha 104. Na linha seguinte, é feita a condicional de forma vetorizada retornando um vetor lógico com TRUE ou FALSE, a função `as.integer()` recebendo esse resultado converte para 1 e 0. Na linha 106 é calculada a quantidade usando a função `sum()` que soma 1 para cada vez que a condição for satisfeita. Por fim, na linha 107 é calculada a proporção. Os resultados exibidos no console ao verificar os objetos foram:

- `Q1_total_user = 3115.5`
- `quantidade = 75`
- `proporcao = 0.25` (ou seja, 1 em 4 dias é de baixo uso)

Esse critério de baixo uso é útil para determinar por exemplo para determinar correlações com outras variáveis, se por exemplo, se uma alta temperatura seria mais comum em dias com baixo uso.

Questão 3

A questão 3 solicita o uso da variável `low_usage` previamente definida na questão 2 para investigar as características associadas aos dias de menor uso do sistema.

Definições

def - Desvio padrão: indica o quão espalhados estão os valores de um conjunto de dados em relação à média, utilizado aqui como desvio padrão amostral, já que estamos analisando uma amostra de 300 valores de uma população total de 731. O desvio padrão amostral é o mesmo utilizado pela função `sd()` do R, dado pela fórmula:

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

def - Coeficiente de correlação de Pearson: mede a intensidade da relação linear entre duas variáveis quantitativas, retornando um valor entre -1 e 1 , dado pela fórmula:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Item 1

Este item pede para calcular média, mediana e desvio padrão de `total_user` para cada estação do ano e determinar a proporção de dias classificados como `low_usage` e, além disso, construir um boxplot comparando cada estação.

Para a resolução do item 1, segue o código em R e a explicação de cada trecho:

```
109 # Questão 3
110 # Item 1
111 # Convertendo season numérico para categórico
112 data_group$season <- factor(data_group$season,
113                             levels = c(1, 2, 3, 4),
114                             labels = c("Inverno", "Primavera",
115                                       "Verão", "Outono"))
116
117 # Média de usuários por estação do ano
118 tapply(total_user, data_group$season, mean)
119
120 # Mediana de usuários por estação
121 tapply(total_user, data_group$season, median)
122
123 # Desvio padrão de total_user por estação
124 tapply(total_user, data_group$season, sd)
125
126 # Proporção de dias classificados como low_usage por estação
127 tapply(low_usage, data_group$season, mean)
```

```

123
124   boxplot(total_user ~ data_group$season,
125           main = "Comparação do uso do sistema por Estação",
126               #Titulo
127           xlab = "Estação do Ano", #legenda eixo x
128           ylab = "Total de Usuários", #legenda eixo y
129           medcol = "#D32F2F",          # Cor da mediana
130           col = c("#81C784", "#FFF176", "#FFB74D", "#64B5F6"),
131               #cores das caixas
132           border = "#2E7D32") #cor da borda da caixa

```

Código 11: Análise da utilização do sistema em cada estação do ano

Primeiro é feita a conversão da variável `season` de numérica para categórica usando `factor` para associar o nome de cada estação ao seu respectivo número (1, 2, 3, 4). A conversão não altera os valores calculados, apenas faz com que os grupos sejam identificados pelo nome da estação, tanto na saída da função `tapply` quanto no eixo do `boxplot`. Depois, é utilizada a função `tapply()` para dividir o vetor de `total_user` em subgrupos, um para cada estação do ano, e aplicar uma função (no caso aqui, as funções de média, mediana e desvio padrão) a cada subgrupo deste vetor. O resultado obtido no R está na tabela a seguir:

Tabela 7: Resultados para cada estação

Medição	Inverno	Primavera	Verão	Outono
Média	2000.76	3775.17	4464.36	3664.46
Mediana	2077.00	4085.50	4615.50	3717.00
Desvio padrão	738.80	1138.90	798.35	983.01
Proporção (<code>low_usage</code>)	0.92	0.26	0.04	0.27

Como a variável `low_usage` é 0 ou 1, fazer a média retorna exatamente a proporção, já que estamos somando os dias classificados como `low_usage` e dividindo pela quantidade de dias.

Para a construção do `boxplot`, foi feita a criação do gráfico fazendo:

```
total_user ~ data_group$season
```

Ou seja, `total_user` em função da variável `season`, gerando uma caixa para cada estação. O `boxplot` gerado está exibido a seguir:

Os resultados evidenciam uma clara associação entre as estações do ano e o uso do sistema, mostrando que o uso diminui consideravelmente no período de inverno e aumenta nas estações mais quentes. Além disso, a extração da proporção de dias classificados como `low_usage` denota que a grande maioria dos dias de inverno é classificada como de baixa utilização do sistema, ademais, o uso do sistema no verão aumenta mais que o dobro do que no inverno, com o verão tendo uma média total de usuários do sistema de 4464.36 contra uma

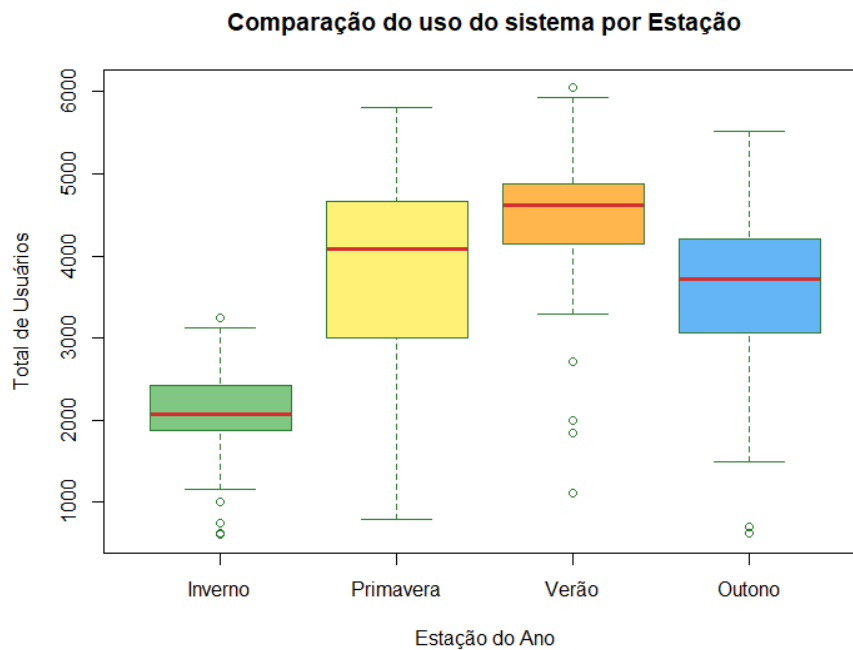


Figura 3: Boxplot do total de usuários por estação

média de 2000.76 usuários no inverno. Outra informação relevante é que as estações de primavera e outono possuem média de usuários diária bem próxima, o que demonstra que o uso do sistema é bem equivalente entre essas duas estações, com uma ressalva de que a primavera tem alto desvio padrão comparado às outras estações, evidenciando uma notável disparidade entre dias de maior uso e de menor uso dentro da própria estação devido a fatores além da estação do ano.

Item 2

Este item busca investigar a relação do uso do sistema em função das condições meteorológicas. Para isso, o item pede que, para cada condição, seja calculada a média e o desvio padrão de `total_user` e determinada a proporção de dias classificados como `low_usage`.

Para isso, foi utilizado o seguinte código em R:

```

132 # Item 2
133 # Convertendo weathersit de numérico para categórico
134 data_group$weathersit <- factor(data_group$weathersit,
135                               levels = c(1, 2, 3, 4),
136                               labels = c("Céu limpo",
137                                         "Nublado", "Chuva fraca",
138                                         "Chuva forte"))
137
138 # Média total_user por condição meteorológica
139 tapply(total_user, data_group$weathersit, mean)

```

```

140
141 # Desvio padrão de total_user por condição meteorológica
142 tapply(total_user, data_group$weathersit, sd)
143
144 # Proporção de dias de baixa utilização (low_usage) por
    condição meteorológica
145 tapply(low_usage, data_group$weathersit, mean)

```

Código 12: Análise da utilização do sistema para cada condição meteorológica

Assim como na resolução do item 1, foi utilizada a seguinte linha de raciocínio: para cada valor correspondente a determinada condição climática constatada no `data_group`, foi associado o nome correspondente da condição climática utilizando `factor`, para facilitar a identificação e interpretação dos dados tanto nos resultados da função `tapply` quanto em representações gráficas. Após isso, usando a função `tapply` para aplicar as funções de média e desvio padrão a cada subgrupo de `total_user` e de `low_usage` (um para cada condição meteorológica), obtiveram-se os seguintes resultados de média, desvio padrão e proporção de dias classificados como baixo uso do sistema, constatados na tabela a seguir:

Tabela 8: Resultados para cada condição meteorológica

Medição	Céu limpo	Nublado	Chuva fraca	Chuva forte
Média	4159.22	3462.94	1757.57	–
Desvio padrão	984.66	1090.92	793.95	–
Proporção (<code>low_usage</code>)	0.14	0.35	1.00	–

Analisando os resultados, nota-se de imediato duas informações cruciais: não se obtiveram dados para a categoria de chuva forte, pois o `data_group` de 300 amostras não possui dias classificados meteorologicamente com essa condição climática; além disso, destaca-se a proporção de dias de baixo uso igual a 1, ou seja, nos dias de chuva fraca da amostra, o total de usuários ficou sempre abaixo do primeiro quartil (Q1) definido para a amostra inteira.

Para realizar uma comparação visual entre as condições meteorológicas, usou-se também um gráfico construído a partir do seguinte código em R:

```

147 # Boxplot
148 boxplot(total_user ~ data_group$weathersit,
149         main = "Utilização do Sistema por Condição
    Meteorológica",
150         xlab = "Condições Meteorológicas",
151         ylab = "Total de Usuários",
152         col = c("#FFF176", "#D3D3D3", "#00A8FF", "#173A59"),
153         border = "#2E7D32")

```

Código 13: Construção de boxplot para análise da utilização do sistema para cada condição meteorológica

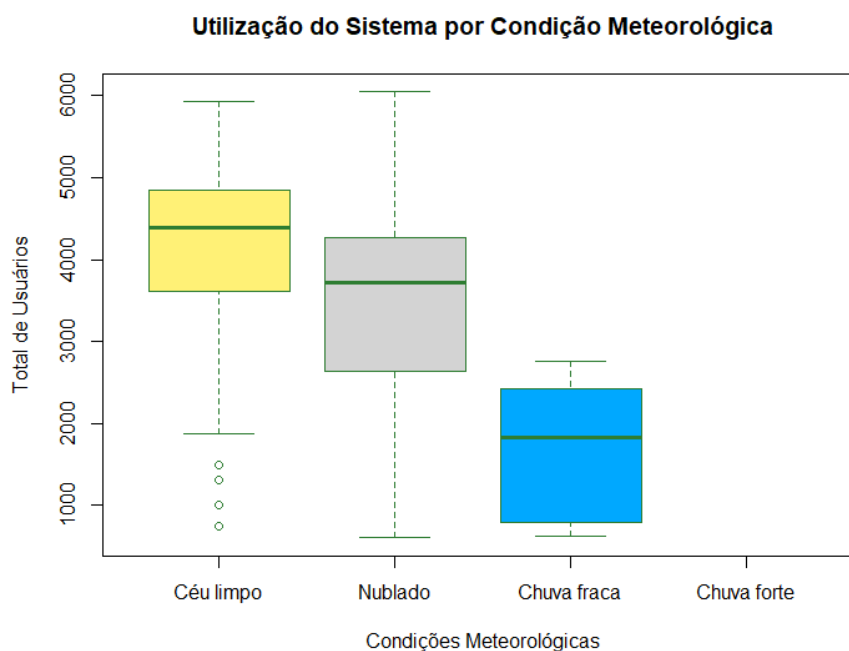


Figura 4: Boxplot do total de usuários por condição meteorológica

Os resultados indicam que a demanda pelo sistema cai de forma expressiva à medida que as condições meteorológicas se deterioram. A média de usuários cai de cerca de 4159, em dias de céu limpo, para apenas 1757, em dias de chuva fraca, o que corresponde a quase 2/3 a menos de usuários do sistema em dias de chuva fraca em comparação com o céu limpo, o que se nota graficamente pela queda em “degraus” das medianas representadas no boxplot. Além disso, observa-se uma imprevisibilidade no uso do sistema em dias nublados, tanto pelo maior desvio padrão quanto pelo tamanho da caixa e dos bigodes no boxplot, mostrando que, mesmo com o clima nublado, há dias de uso bastante elevado do sistema, enquanto outros igualam o nível de uso em dias chuvosos, denotando a clara variabilidade no comportamento dos usuários frente a essa condição climática.

Item 3

Este item busca investigar a relação entre temperatura e `total_user`, usando para isso um gráfico de dispersão e o coeficiente de correlação para descrever a relação observada. Para isso, foi utilizado o seguinte código em R:

```

155     #item 3
156     # Cálculo do coeficiente de correlação de Pearson
157     correlacao <- cor(data_group$temp, total_user)
158     print(correlacao)
159
160     # Construção do gráfico de dispersão
161     plot(data_group$temp, total_user,
```

```

162     main = "Relação entre temperatura e utilização do sistema",
163     xlab = "Temperatura em graus Celsius",
164     ylab = "Total de Usuários",
165     pch = 19,                                # Formato do ponto (bolinha
        preencheda)
166     col = "#2E7D32",                          # Cor dos pontos
167     frame.plot = FALSE)                        # Remove a moldura externa
        superior e direita

```

Código 14: Obtenção do coeficiente de correlação de Pearson e plotagem do gráfico de dispersão

Primeiro é extraído diretamente o coeficiente de correlação (que no R, por padrão, é o de Pearson) através da função `cor(data_group$temp, total_user)`, correlacionando a temperatura e o `total_user`.

O resultado obtido no terminal foi $r = 0.6386851$, o que denota uma correlação positiva e moderada-alta, ainda distante de 1, o que indica que, com o aumento da temperatura, o número de usuários do sistema tende a aumentar, entretanto com uma considerável dispersão dos pontos no gráfico, indicando que outros fatores também influenciam nessa relação.

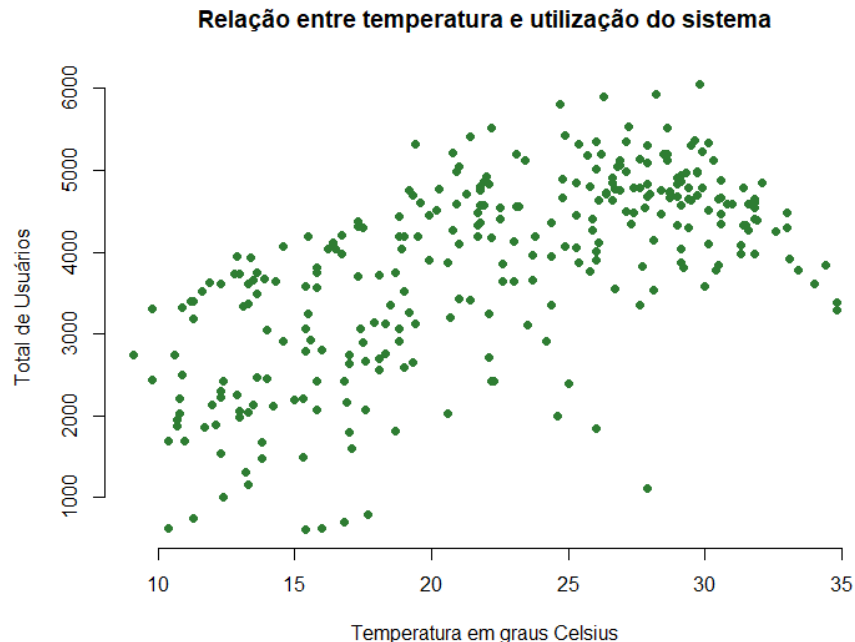


Figura 5: Gráfico de dispersão (`total_user` x temperatura)

Evidenciam-se, portanto, algumas limitações do coeficiente de Pearson para descrever essa relação. Por exemplo, o r de Pearson é estritamente linear, o que reduz sua precisão em relações não lineares: nota-se um crescimento quase linear até por volta dos 30°C, e depois disso o total de usuários começa a cair com o aumento da temperatura, o que formaria uma

curva em parábola, relação que o coeficiente linear não captura. Além disso, por natureza, correlação não implica causalidade, e também é sensível a outliers, como pontos isolados de baixo uso do sistema bem abaixo do padrão da vizinhança, que tende a ser mais alto.

Item 4

Este item pede para escolher 2 das 3 categorias analisadas previamente nesta questão que parecem estar mais associadas à utilização do sistema. Baseado nos dados obtidos e na análise feita, as duas categorias escolhidas serão a condição meteorológica e a temperatura, pois, apesar da estação do ano influenciar a demanda do sistema, ela é uma categoria que engloba em partes características das outras duas. Por exemplo, no item 1 observou-se que, no inverno, a demanda pelo sistema cai consideravelmente; entretanto, o inverno afasta as pessoas justamente pelo fato de ter temperaturas baixas e condições meteorológicas mais turbulentas, e nos itens 2 e 3 foi comprovado que condições meteorológicas piores e temperaturas baixas têm, no geral, baixa adesão dos usuários ao sistema. Feita essa escolha das 2 categorias, os resultados estatísticos mostram o seguinte sobre elas:

Condições meteorológicas:

Observou-se que esta categoria é um fator altamente impactante na demanda do sistema, visto que a média de usuários despencou de 4159, em dias de céu limpo, para somente 1757, em dias de chuva fraca. Além disso, através da análise da proporção de dias classificados como `low_usage`, 100% dos dias com chuva fraca registraram baixa utilização do sistema, provando ser um fator drástico que limita a quantidade de usuários que aderem ao sistema de aluguel de bicicletas, sendo assim uma categoria intensamente associada à utilização do sistema.

Temperatura:

Os resultados mostram que a temperatura possui considerável relação com a utilização do sistema. Através do coeficiente de correlação de Pearson obtido ($r \approx 0.64$), evidencia-se uma forte associação linear positiva, com o gráfico de dispersão ilustrando claramente o aumento gradual no número de usuários conforme a temperatura aumenta, até um limite de conforto e saúde por volta de 30-32°C. Além disso, o gráfico de dispersão, apesar de estar analisando a relação temperatura x total de usuários, também reforça a associação encontrada no item 1 entre estação do ano e `total_user`, constatando-se que estações do ano mais frias apresentam médias de usuários menores do que estações mais quentes, com a média de usuários do inverno (2000) sendo menos da metade da média de usuários do verão (4464).

Questão 4

A questão 4 pede uma análise mais profunda sobre o fenômeno de baixa utilização do compartilhamento de bicicletas e sua relação com a temperatura e estações do ano, como investigado nas questões anteriores.

Item 1

O item pede para que seja feita uma série temporal no R do número total de usuários no decorrer do período registrado no dataframe.

Utilizando o número total de usuários evidenciado pela variável `total_user`, é criada uma variável do tipo série temporal (ts) de nome `sdata` para representar os dias em que foram registrados os usuários. Ela é definida por um começo da contagem de tempo pelo dia inicial 2011/01/01 (ano/mês/dia), conforme mostrado no dataframe e um fim não definido, onde faria a contagem terminar no ultimo dado, em 2012/12/31, também é estabelecido uma frequência de 365,65 que correspondem os dias por ano (incluindo bissexto).

O código abaixo mostra o comando utilizado:

```
169   # Questão 4
170   #item 1: criação de variável sdata como série temporal com
        base de total_user
171
172   sdata <- ts(total_user, start = c(2011, 1), frequency = 365.65)
```

Código 15: Definição de variável

Com a série temporal definida, será necessário mostrar a representação gráfica (plot) do número de usuários e mostrar o ponto mínimo e máximo de registro de usuários. Cria-se um plot com `sdata` com a nomeação de título, coluna e linha, mostrando o total de usuários em relação com o período de tempo (nos 2 anos de registro do dataframe), respectivamente.

```
174   #uso do plot para representação gráfica,
175   #main: titulo
176   #ylab: texto do eixo y
177   #xlab: texto do eixo x
178   plot(sdata,
179         main = "série temporal",
180         ylab = "total de usuários",
181         xlab = "tempo")
```

Código 16: Plot da série temporal

Para encontrar os pontos mínimo e máximo dentro do plot, defina-se dois pontos de busca no `total_user`, um com o menor número de registros de compartilhamento e outro com o maior, seguido da data correspondente. Além disso, para melhor compreensão, esses pontos serão marcados para facilitar a localização na representação gráfica, na qual a coluna com o ponto de cor vermelha sinaliza o período onde teve o menor registro de compartilhamento, e a coluna com o ponto de cor azul o período onde teve mais registros.

```
174   #para melhor compreensão do gráfico
```

```

175 #acrescentando pontos de mínimo (vermelho) e máximo (azul) no
    gráfico com os respectivos número de usuários e dia do
    registro
176
177 points(time(sdata)[which.min(sdata)],
178        min(sdata),
179        col = ("red"),
180        pch = 19) #cor e espessura do ponto mínimo
181 text(time(sdata)[which.min(sdata)],
182       min(sdata),
183       labels = paste0("mín: ", min(sdata), " - ",
184                      data_group$dteday[which.min(sdata)]),
185       col = ("red"),
186       pos = 4,) #cor e posicionamento do texto do ponto mínimo
187
188 points(time(sdata)[which.max(sdata)],
189        max(sdata),
190        col = ("blue"),
191        pch = 19) #cor e espessura do ponto máximo
192
193 text(time(sdata)[which.max(sdata)],
194       max(sdata),
195       labels = paste0("máx: ", max(sdata), " - ",
196                      data_group$dteday[which.max(sdata)]),
197       col = ("blue"),
198       pos = 4) #cor e posicionamento do texto do ponto máximo

```

Código 17: Plot da série temporal

A representação gráfica obtém o seguinte resultado utilizando as 300 observações:

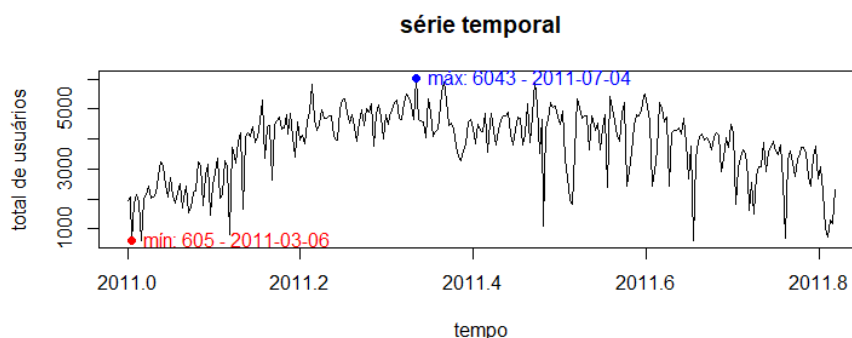


Figura 6: Série temporal)

Item 2

Este item busca retomar as duas características selecionadas na questão 3 e identificar uma visualização e medida estatística adequada.

Para temperatura, a visualização pelo gráfico de dispersão (5), que já foi feito, provavelmente é melhor escolha, pois relaciona duas variáveis e permite verificar a causalidade e a correlação de forma fácil. Além disso, como a tendência linear é perceptível, o coeficiente de pearson é adequado para identificar a relação.

Para condições meteorológicas, o box-plot também é a melhor escolha para visualização. Mas também foi feito um gráfico de pizza para associar a quantidade total de usuários e as condições meteorológicas. O código utilizado foi o seguinte:

```
207   #item 2:
208
209   #declarando variável de registro de temperatura e condição,
      classificando o tipo de condição meteorológica
210   d_temp <- c(data_group$temp) #dados temperatura
211
212   d_met <- c(data_group$weathersit) #dados condição meteorologica
213   nivel_met <- factor(d_met, levels = c(1, 2, 3), labels =
      c("ceu limpo", "nublado", "chuva fraca"))
214
215   #criação de gráfico (pizza) para relação com condição e
      usuários
216   pie(table(nivel_met),
217       col = c("#B8B8B8", "#808080", "#404040"),
218       main = "Gráfico de Pizza - Níveis",)
```

Código 18: Gráfico de pizza

Segue imagem resultante:

Gráfico de Pizza - Níveis

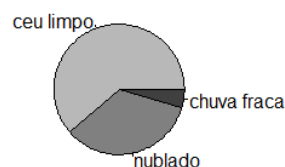


Figura 7: Gráfico de pizza

Item 3

Este item solicitava para fazer uma análise relacional entre temperatura e `total_user`, diferenciando visualmente os dias classificados como `low_usage`.

A partir das 300 observações e como observado na questão 3, os usuários totais apresentam uma correlação positiva com a temperatura. Para identificar visualmente todos os dias classificados como `low_usage`, é feita uma modificação no gráfico de dispersão (5), determinando uma condição nas cores, que se for um dia de baixa utilização será vermelho e caso contrário verde escuro.

```
220   #item 3:
221   #criação do gráfico (dispersão), com condições para
      identificar dias de baixo uso
222   cores <- ifelse(low_usage == 1, "red", "#2E7D32")
223   plot(data_group$temp, total_user,
224         main = "Relação entre temperatura e utilização do sistema
      (com dias de baixo uso)",
225         xlab = "Temperatura em graus Celsius",
226         ylab = "Total de Usuários",
227         pch = 19,                      # Formato do ponto (bolinha
      preenchida)
228         col = cores,                    # Cor dos pontos
229         frame.plot = FALSE)            # Remove a moldura externa
      superior e direita
```

Código 19: Dispersão e Low Usage

A imagem resultante foi:

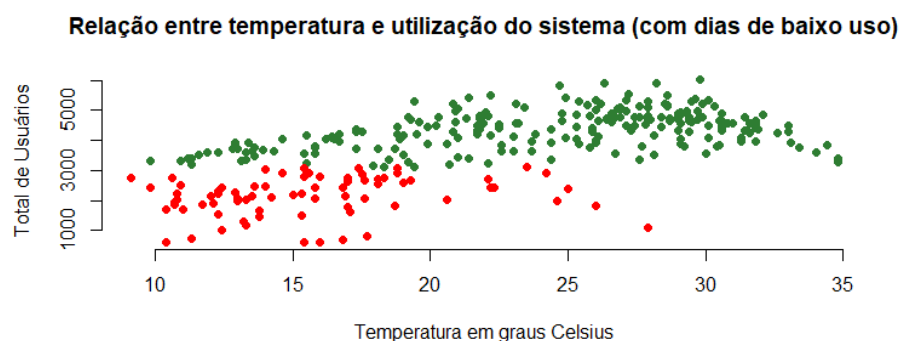


Figura 8: Gráfico com low usage evidenciado

Item 4

A análise comparativa entre a quantidade diária do `total_user` e as características permite concluir que há uma correlação direta entre elas. Por exemplo, no box plot (4) é conclusivo que em dias de céu limpo ou nublado há mais usuários, comparando com dias de chuva fraca. Também é possível afirmar que quanto maior a temperatura maior o número de usuários, apresentando uma correlação positiva, conforme o gráfico (5). A influência da estação para os usuários é a menor entre as características, por que a única que possui diferença significativa em comparação com as outras é o inverno, no entanto, essa estação é a que apresenta temperaturas menores, logo é inferível que a correlação positiva decorre da temperatura e não das outras características associadas da estação.